Published: 08 August 2016
Accepted: 01 May 2016
Revised: 01 December 2015
Received: 01 August 2014

RESEARCH-ARTICLE

Simple Parallel and Distributed Algorithms for Spectral Graph Sparsification

IOANNIS KOUTIS, University of Puerto Rico at Rio Piedras, San Juan, Puerto Rico

SHENCHEN XU, Carnegie Mellon University, Pittsburgh, PA, United States

Simple Parallel and Distributed Algorithms for Spectral Graph Sparsification

IOANNIS KOUTIS, Computer Science Department, University of Puerto Rico-Rio Piedras
SHEN CHEN XU, Computer Science Department, Carnegie Mellon University

We describe simple algorithms for spectral graph sparsification, based on iterative computations of weighted spanners and sampling. Leveraging the algorithms of Baswana and Sen for computing spanners, we obtain the first distributed spectral sparsification algorithm in the CONGEST model. We also obtain a parallel algorithm with improved work and time guarantees, as well as other natural distributed implementations. Combining this algorithm with the parallel framework of Peng and Spielman for solving symmetric diagonally dominant linear systems, we get a parallel solver that is significantly more efficient in terms of the total work.

Categories and Subject Descriptors: F.2 [**Analysis of Algorithms and Problem Complexity**]

General Terms: Parallel Algorithms, Distributed Algorithms

Additional Key Words and Phrases: Spectral sparsification, SDD linear systems, sparsest cut

ACM Reference Format:
Ioannis Koutis and Shen Chen Xu. 2016. Simple parallel and distributed algorithms for spectral graph sparsification. ACM Trans. Parallel Comput. 3, 2, Article 14 (August 2016), 14 pages.
DOI: http://dx.doi.org/10.1145/2948062

1. INTRODUCTION

The efficient transformation of dense instances of graph problems to nearly equivalent sparse instances is a powerful tool in algorithm design. Spectral sparsifiers are sparse graphs that preserve within a $(1+\epsilon)$ factor the quadratic form $x^T L_G x$, where L_G is the Laplacian of G and ϵ is a parameter of choice. They were introduced by Spielman and Teng [2004] as a basic component of the first nearly linear time solvers for linear systems on symmetric diagonally dominant (SDD) matrices.[1] Such linear system solvers are a key algorithmic primitive with numerous applications [Koutis et al. 2012; Teng 2010].

The Spielman and Teng sparsification algorithm produces sparsifiers with $O(n \log^c n/\epsilon^2)$ edges for some fairly large constant c, where n is the number of vertices in the graph. At a high level, their algorithm is based on graph decompositions into edge-disjoint sets that get sparsified independently via uniform sampling. As noted in Peng and Spielman [2014], the algorithm can be parallelized if the original partitioning subroutine is substituted by a more recent one by Orecchia and Vishnoi [2011].

[1]A symmetric matrix A is SDD if for all i, $A_{ii} \geq \sum_{j \neq i} |A_{ij}|$.

This work was supported by NSF CAREER award CCF-1149048. Part of this work was done while the author was visiting the Institute of Computational and Experimental Mathematics (ICERM) in Spring 2014.
Authors' addresses: I. Koutis, Department of Computer Science, College of Natural Sciences, University of Puerto Rico, Rio Piedras Campus, PO Box 70377, San Juan, PR 00936–8377; email: ioannis.koutis@upr.edu; S. C. Xu, Computer Science Department, Carnegie Mellon University, 5000 Forbes Ave, Pittsburgh, PA 15213; email: shenchex@cs.cmu.edu.

DOI: http://dx.doi.org/10.1145/2948062

Peng and Spielman [2014] recently presented a novel algebraic framework for solving SDD linear systems. It enables the use of parallel sparsification algorithms for constructing parallel solvers. Combined with the parallelized Spielman and Teng sparsification algorithm, or a more recent approach due to Peng (Section 3.4, Peng [2013]), this algebraic framework yields the first "truly" parallel SDD solver that does near-linear work and runs in polylogarithmic time.

The new parallel solver leaves something to be desired: Its work is by several logarithmic factors larger than that of the fastest known sequential algorithm that runs in $\tilde{O}(m \log n)$ time[2]; here m is the number of non-zero entries in the matrix [Koutis et al. 2011]. This motivates our study on parallel and distributed sparsification algorithms.

1.1. Background on Spectral Sparsification and Related Work

Besides yielding the SDD solver, the work of Spielman and Teng ignited further research on spectral sparsification as a stand-alone problem. Spielman and Srivastava [2008] showed that it is possible to produce a sparsifier with $O(n \log n/\epsilon^2)$ edges in near-linear time. Their approach is based on viewing the graph as an electrical resistive network, where one can define the effective resistance of an edge as the potential difference that must be applied between its two endpoints in order to send one unit of electrical flow from the one vertex to the other. The sparsifier is computed by sampling edges with probabilities proportional to the their effective resistances. Spielman and Srivastava also showed that $O(\log n)$ calls to a solver for SDD linear systems can produce sufficiently good approximations to all effective resistances, allowing for a near-linear time implementation of their sampling scheme. This development was followed by works on slower but more sparsity-efficient spectral sparsification algorithms [Batson et al. 2009; Kolla et al. 2010] and on sparsification in the semi-streaming model [Kelner and Levin 2011].

The work of Spielman and Srivastava opened the way to the near-$m \log n$ time solver in Koutis et al. [2011, 2014]. This fast solver utilizes an "incremental sparsification" algorithm that produces a very mildly sparser spectral approximation to the input graph. A direct by-product of this fast solver was the acceleration of the Spielman-Srivastava sparsification scheme. Their scheme was further improved in Koutis et al. [2012a, 2012b], yielding an $\tilde{O}(m)$ solver for slightly non-sparse graphs; the solver combines in an intricate recursive way slower solvers with spectral sparsifiers.

Recent efforts aim to obtain simpler algorithms via alternative approaches. In particular, there has been an interest in combinatorial algorithms that rely less on the power of algebra to achieve similar results [Kapralov and Panigrahy 2012; Kelner et al. 2013]. We do not insist that these simpler algorithms are asymptotically as efficient as their algebraic counterparts. In practice, there are many phenomena, subtler than asymptotic behavior or even hidden constants, that affect the performance of linear system solvers, and different ideas may lead to better implementations. In particular, there are implementations that exhibit great empirical performance on sparse matrices [Koutis and Miller 2009; Livne and Brandt 2012]; solve-free techniques for spectral sparsification have the potential of extending the applicability of these implementations to dense matrices.

The first combinatorial alternative to the spectral sparsification algorithm of Spielman and Teng was given by Kapralov and Panigrahy [2012]. A novel feature of their work is the introduction of spanners in the context of spectral graph sparsification. The algorithm is based on tightly approximating effective resistances; more concretely, they define "robust connectivities" of edges and show they are good upper bounds to the

[2]We use $\tilde{O}()$ to hide a $poly(\log \log n)$ factor.

effective resistances, on average. Approximate robust connectivities are then used for sampling; the result follows from an application of the "oversampling" lemma of Koutis et al. [2014] that shows that extra sampling can compensate for the the lack of accuracy in the estimates for the effective resistances; this extra sampling yields the slightly more dense sparsifier. The algorithm generates a sparsifier with $O(n \log^4 n/\epsilon^4)$ edges in $O(m \log^4 n)$ time but it does not parallelize mostly due to the use of distance oracles by Thorup and Zwick [2005].

For a more thorough review of the sparsification literature, we refer the reader to the excellent article by Batson et al. [2013].

1.2. Our Contributions

We describe a simple combinatorial algorithm that exposes a cleaner connection between spanners and spectral sparsification, improving on all previous combinatorial sparsification algorithms. The algorithm lends itself to work-efficient implementations:

(a) In the well-known Concurrent Read Concurrent Write Parallel RAM (CRCW PRAM) model [Kumar 2002].
(b) In the CONGEST distributed model, where the computational network is identical to the input graph [Pandurangan and Khan 2010]. Nodes communicate in synchronous rounds. In each round, every node exchanges one message of size $O(\log n)$ with each of its neighbors. The communication complexity is the total number of messages.
(c) In the practical distributed setting when k machines are available and the goal is balance the work load while minimizing the communication with a central coordinator. In the proposed solution the coordinator implements an elementary streaming algorithm that simply reads edges and distributes them to the k machines.

The main idea behind our approach is to reduce sparsification to a *half-sparsification* problem, in which the desired output is guaranteed to be sparser only by a factor of 2, while being a $(1 + \epsilon/\log \rho)$-approximation. The constant sparsification subroutine can then be invoked $O(\log \rho)$ times to obtain a $(1 + \epsilon)$-approximation that is sparser by a factor of ρ, where ρ is a parameter of choice. This idea is described in Section 3.3.

In Section 3.1, we describe our first half-sparsification algorithm. The key idea is to iteratively "peel-off" spanners from the input graph. The observation is that the removal of $O(\log^2 n \log^2 \rho/\epsilon^2)$ spanners from the graph allows us to certify upper bounds for the effective resistances of the rest of the edges. The upper bounds enable us to half-sparsify the graph via uniformly sampling the remaining edges. This algorithm is used to establish our claims about the CRCW PRAM and the CONGEST models. In Section 3.2, our first algorithm is leveraged to obtain a slightly different half-sparsification algorithm which is used in the k-machine distributed algorithm.

Using either of these two subroutines, the total work of our sparsification algorithm is $O(mw \log^2 n \log^2 \rho/\epsilon^2)$, where $w = 1$ or $w = \log n$ depending on which spanner computation algorithm is used. This holds for all proposed implementations.

In Section 3.4 we provide a new combinatorial algorithm for further sparsifying the sparse output of our main sparsification algorithm. We obtain the algorithm by taking a closer look at the spanner peeling-off process, continued until the graph is exhausted. The process yields upper bounds for the effective resistance of the edges that are peeled off along the way. These bounds get progressively better and they can be used to compute a sparsifier with nearly $O(n \log^2 n/\epsilon^2)$ edges, which is the sparsest output known to be achievable by a combinatorial algorithm.

In Section 4 we discuss the implication of our sparsification algorithms for SDD linear system solvers. We obtain a solver that works in polylogarithmic time and does

$\tilde{O}((m\log^2 n + n\log^3 n \log^5 \kappa)(\log(1/\tau))$ work, where τ is a standard measure of tolerance in the error of the approximate solution, and κ is the condition number of the input system.

2. BACKGROUND

Laplacians. Given a weighted graph $G = (V, E, w > 0)$ where $V = \{1, \ldots, n\}$, its Laplacian L_G is the matrix defined by

$$\text{(i) } L_G(i,j) = -w_{ij} \text{ for } i \neq j \text{ and } \quad \text{(ii) } L_G(i,i) = \sum_{j \neq i} w_{ij}.$$

Throughout the article we will use n, m to denote the number of vertices and edges of a graph, respectively. We will apply algebraic operators on graphs in a standard way. Specifically, given two graphs $G_1 = (V, E, w_1)$ and $G_1 = (V, E, w_2)$ we denote by $G_1 + G_2$ the graph $(V, E, w_1 + w_2)$. Also, given a scalar a, we let $aG_1 = (V, E, aw_1)$.

Spectral approximation. We say that a graph H, (β/α)-approximates a graph G if:

$$\alpha(x^T L_H x) \leq x^T L_G x \leq \beta(x^T L_H x).$$

Finally, if for all vectors x we have $x^T L_{G_2} x \leq x^T L_{G_1} x$ we will write $G_2 \preceq G_1$.

Stretch. Let p be a path joining the two endpoints of an edge $e \in E$. The stretch $st_p(e)$ of an edge e, is equal to

$$w_e \sum_{e' \in p} (1/w_{e'}).$$

We also define the stretch of e over a graph H as

$$st_H(e) = \min_{p \in H} st_p(e).$$

Spanners. A $\log n$-spanner of a graph G is a subgraph H of G such that for all edges $e \in E$

$$st_H(e) \leq 2\log n.$$

In the rest of the article, we will use the term spanner to mean a $\log n$-spanner.

Definition 2.1. Let G be a graph and $H_1, \ldots, H_t$ be subgraphs of G such that H_i is a spanner for the graph $G - \sum_{j=1}^{i-1} H_j$. We call $H = \sum_{j=1}^{t} H_j$ a t-bundle spanner. We call the H_i's the components of H. When $G = H$, we refer to H as a t-bundle decomposition of G.

Parallel *t*-Bundle Spanner Construction. A t-bundle spanner can be computed iteratively in the obvious way: In the ith iteration, we compute a spanner H_i for $G - \sum_{j=1}^{i-1} H_j$. Edges in $\sum_{j=1}^{i-1} H_j$ can declare themselves out of the ith iteration in the parallel or distributed model. Thus extending existing spanner construction algorithms is easy. There are various known algorithms for computing spanners, with slightly different guarantees, that are subject for future improvement. For this reason, we will use throughout the article the following symbols:

(i) s_d to denote the size of the spanner output by an algorithm implemented in the CONGEST model, c_d the communication complexity of the algorithm, and t_d the number of rounds.

(ii) s_p to denote the size of the spanner output by an algorithm implemented in the CRCW PRAM model, w_d the total work of the algorithm, and t_p the parallel time. We will be using randomized algorithms that achieve the size guarantee in expectation, with high probability.

We will just use s for the spanner size, understood to mean s_p or s_d depending on the context. Currently known upper bounds for these parameters are as follows:

(a) $s_d = O(n \log n)$, $c_d = O(m \log n)$ and $t_d = O(\log^2 n)$ [Baswana and Sen 2007].
(b) $s_p = O(n \log n)$, $w_p = O(m \log n)$ and $t_p = O(\log n \log^* n)$ [Baswana and Sen 2007].
(c) $s_p = O(n \log \log n)$, $w_p = O(m)$, $t_p = O(\log U \log n \log^* n)$, where U is the ratio of the maximum to the minimum graph weights [Miller et al. 2015].

With this terminology, we get the following lemmas.

LEMMA 2.2. *On input of a graph G, a t-bundle spanner for G of expected size $O(ts_p)$ can be constructed with $O(tw_p)$ work in $O(tt_p)$ time in the CRCW PRAM model.*

LEMMA 2.3. *On input of a graph G, a t-bundle spanner for G of size $O(ts_d)$ can be constructed in the CONGEST model in $O(tt_d)$ rounds and $O(tc_d)$ communication complexity.*

Effective Resistance. A graph can be viewed as an electrical resistive network, with each edge corresponding to a resistor having resistance $r_e = 1/w_e$. The effective resistance $R_{u,v}[G]$ between two vertices u and v in G is defined as the potential difference that has to be applied on u and v in order to drive one unit of current through the network. For instance, in the case of a path p the effective resistance between the two endpoints of p is equal to $R_e[p] = \sum_{e' \in p}(1/w_{e'})$; this is the well-known formula for resistors connected in series.

Now let us recall a simple fact about paths connected "in parallel," that is, paths that are vertex-disjoint with the exception of their shared endpoints u and v. Let $p_1, \ldots, p_t$ be paths connected in parallel. Let $P = \sum_{i=1}^{t} p_i$. For the effective resistance between u and v, in the graph P consisting of the union of the paths, we have

$$R_{u,v}[P] = \left(\sum_{i=1}^{t} (R_{u,v}[p_i])^{-1} \right)^{-1}. \tag{1}$$

The following lemma has a key role in our sparsification algorithm.

LEMMA 2.4. *Let G be a graph and H be a $2t$-bundle spanner of G. For every edge e of G which is not in H, we have*

$$w_e R_e[G] \leq \log n / t.$$

PROOF. Let $H_1, \ldots, H_{2t}$ be the components of H. If H' is any subgraph of G, then by Rayleigh's monotonicity law [Doyle and Snell 2000] the effective resistance of e is at most equal to the effective resistance between the two endpoints of e in H'. In particular, fix an arbitrary edge e not in H. For each i, we know by definition that it contains a path p_i such that

$$w_e \sum_{e' \in p_i} (1/w_{e'}) \leq 2 \log n.$$

As we discussed above, $\sum_{e' \in p_u}(1/w_{e'})$ is equal to the resistance between the two endpoints of e in p. This implies that the effective resistance of e over p_i satisfies

$$R_e[p_i] \leq 2 \log n / w_e.$$

Now we observe that by definition the paths p_i connect in parallel the two endpoints of e. Let $P = \sum_{j=1}^{t} p_i$. By invoking equality 1 and combining with the last inequality we get that

$$(R_e[P])^{-1} = \left(\sum_{i=1}^{t}(R_e[p_i])^{-1}\right) \geq tw_e/\log n,$$

which implies

$$R_e[P] \leq \log n/(tw_e).$$

Finally, we have $R_e[G] \leq R_e[P]$ by Rayleigh's monotonicity law, since P is a subgraph of G. □

Let B_e be the $n \times n$ Laplacian of the unweighted edge e (which is zero everywhere except a 2×2 submatrix). Looking at the effective resistance algebraically, it is well understood (e.g., see Spielman and Srivastava [2008]) that:

$$B_e \preceq R_e[G]G.$$

Then the above lemma implies the following.

COROLLARY 2.5. *Let G be a graph and H be a $2t$-bundle spanner of G. For every edge e of G that is not in H, we have*

$$w_e B_e \preceq \frac{\log n}{t} G.$$

3. COMBINATORIAL SPARSIFICATION

3.1. Half-Sparsification

We will sparsify graphs using sampling. The Spielman-Srivastava scheme fixes the number of samples and for each sample one edge is selected according to a fixed probability distribution and gets added to the sparsifier [Spielman and Srivastava 2008]. In Algorithm 1 we use a different sampling scheme, sampling each edge independently with a fixed probability.

ALGORITHM 1: HALFSPARSIFY

Input: Graph G, parameter ϵ
Output: Graph $\tilde{G}$
Compute a $(48 \log^2 n/\epsilon^2)$-bundle spanner H for G;
Let $\tilde{G} := H$;
for *each edge* $e \notin H$ **do**
  Add e to $\tilde{G}$ with probability $1/4$ and weight $4w_e$
end
Return $\tilde{G}$;

We will need a theorem by Tropp [2012] and, more specifically, its following variant [Harvey 2012].

THEOREM 3.1. *Let $Y_1, \ldots, Y_k$ be independent positive semi-definite matrices of size $n \times n$. Let $Y = \sum_i Y_i$. Let $Z = E[Y]$. Suppose $Y_i \preceq RZ$. Then, for all $\epsilon \in [0, 1]$,*

$$Pr\left[\sum_i Y_i \preceq (1-\epsilon)Z\right] \leq n \cdot exp(-\epsilon^2/2R)$$

$$Pr\left[\sum_i Y_i \succeq (1+\epsilon)Z\right] \leq n \cdot exp(-\epsilon^2/3R).$$

We have the following theorem.

THEOREM 3.2. *The output $\tilde{G}$ of algorithm* HALFSPARSIFY *on input G and ϵ satisfies with probability $1 - 1/n^2$ the following:*

(a) $(1-\epsilon)G \preceq \tilde{G} \preceq (1+\epsilon)G$.
(b) *The expected number of edges in $\tilde{G}$ is at most*

$$O(st + m/2).^3$$

Here $t = O(\log^2 n/\epsilon^2)$. HALFSPARSIFY *can be implemented in the CRCW PRAM model and the CONGEST model with work, parallel time, and communication guarantees dominated by those of computing a t-bundle spanner, provided by Lemmas 2.2 and 2.3.*

PROOF. The work, parallel time, and communication complexity guarantees for HALFSPARSIFY follow directly. Now let B_e be the $n \times n$ Laplacian of the unweighted edge e. For each edge $e \notin H$, we let Y_e be the random variable defined as follows:

$$\begin{aligned} Y_e &= 0, && \text{with probability } 3/4, \\ &= 4w_e B_e && \text{with probability } 1/4. \end{aligned}$$

Also we let

$$H_i = \lfloor \epsilon^2/(6\log n)\rfloor H,$$

for $i = 1, \ldots, (\lfloor \epsilon^2/(6\log n)\rfloor)^{-1}$. We apply Theorem 3.1 to the random matrix that is formed by summing the H_i's and the Y_e's. For the output of the algorithm, we clearly have

$$\tilde{G} = \sum_{e\notin H} Y_e + \sum_i H_i = \sum_{e\notin H} Y_e + H.$$

We also have that $E[\tilde{G}] = G$. Using $H \preceq G$, for each i we have

$$H_i = \lfloor \epsilon^2/(6\log n)\rfloor H \preceq \epsilon^2/(6\log n)G.$$

In addition, for each $e \notin H$, we have

$$Y_i \preceq 4w_e B_e \preceq \epsilon^2/(6\log n)G.$$

The last inequality follows by setting $t = 48\log^2 n/\epsilon^2$ in Corollary 2.5. Thus the condition of Theorem 3.1 is satisfied for $R = \epsilon^2/(6\log n)$, which, substituted in the bounds of the theorem, proves that (a) holds with probability at least $1 - 1/2n^2$. For (b), observe that the expected number of edges in H is $O(s\log^2 n/\epsilon^2)$ as stated in Lemmas 2.2 and 2.3. The expected numbers of edges outside H is $m/4$ and a simple application of Chernoff's inequality implies that the number is at most $m/2$ with probability at least $1 - 1/2n^2$. Hence, a union bound gives that both (a) and (b) hold with probability at least $1 - 1/n^2$. □

[3]Recall that we will be using s to denote the spanner size.

3.2. Streaming Half-Sparsification

We now describe an alternative constant sparsification algorithm. Below we will use a slightly modified version of HALFSPARSIFY that succeeds with probability equal to $1 - 1/n^3$ and reduces the graph size by a factor of 4; the only needed modification is to increase by a factor of 2 the number of spanners in the bundle computed by the algorithm in the previous section.

ALGORITHM 2: HALFSPARSIFY-EXT

Input: Graph G, parameter ϵ
Output: Graph $\tilde{G}$
/* $S(n, \epsilon)$ is an explicitly known function in $O(s \log^2 n/\epsilon^2)$ */
Arbitrarily split the edges of G into edge-disjoint graphs G_i of size $S(n, \epsilon)$;
For each G_i let $\tilde{G}_i :=$ HALFSPARSIFY(G_i, ϵ);
Return $\tilde{G} = \sum_i \tilde{G}_i$;

THEOREM 3.3. *The output $\tilde{G}$ of algorithm* HALFSPARSIFY-EXT *on input G and ϵ satisfies with probability $1 - 1/n^2$ the following:*

(*a*) $(1 - \epsilon)G \preceq \tilde{G} \preceq (1 + \epsilon)G$.
(*b*) *The expected number of edges in $\tilde{G}$ is at most $m/2$.*

The algorithm has the work guarantees of Theorem 3.2.

PROOF. The main observation in that HALFSPARSIFY reduces the size of the graph by a factor of 2, as soon as the size of the graph is at least some explicitly known function $S(n, \epsilon)$ in $O(sn \log^2 n/\epsilon^2)$. Properties (a) and (b) hold for each G_i with probability at least $1 - 1/n^3$. Via a union bound this implies the properties for the entire graph, with probability $1 - 1/n^2$. In particular, (b) follows by a well-understood fact known as the splitting lemma [Boman and Hendrickson 2003]. The total work for splitting the edges is $O(m)$. The total work in each call of HALFSPARSIFY is dominated by finding the t-bundle; summing over the subgraphs gives a total work proportional to that of finding the t-bundle in the entire graph. □

In a setting where there are k machines, a central coordinator can almost evenly split the load to the k machines in a straightforward streaming fashion; this is because the splitting can be arbitrary. Each machine needs only $O(S(n, \epsilon))$ working memory to compute its output and can simply stream it to the coordinator. The coordinator can re-stream the edges it receives back to the k machines for further rounds of HALFSPARSIFY-EXT.

Notice that the call to the combinatorial HALFSPARSIFY can be replaced by a call to an algebraic sparsification algorithm [Spielman and Srivastava 2008]. In this case, the threshold function $S(n, \epsilon)$ and the required working memory for each machine is $O(n \log n/\epsilon^2)$.

3.3. The Algorithm

The main sparsification routine is presented in Algorithm 3.

We prove the following theorem.

THEOREM 3.4. *The output $\tilde{G}$ of algorithm* SPARSIFY *on input G and ϵ, ρ satisfies*

$$(1 - \epsilon)G \preceq \tilde{G} \preceq (1 + \epsilon)G$$

with high probability. The expected number edges in G is at most

$$O(st + m/\rho).$$

ALGORITHM 3: SPARSIFY

Input: Graph G, parameters ϵ, ρ
Output: Graph $\tilde{G}$
Set $G_0 := G$;
for $i = 1 : \lceil \log \rho \rceil$ **do**
  Set $G_i :=$ HALFSPARSIFY($G_{i-1}, \epsilon/\lceil \log \rho \rceil$)
end
Return $G_{\lceil \log \rho \rceil}$;

Here, $t = O(\log^2 n \log^2 \rho/\epsilon^2)$. The algorithm can be implement in the CRCW model with $O(tw_p)$ work, in $O(tw_t \log \rho)$ time. The algorithm can also be implemented in the CONGEST model to run in $O(tw_d \log \rho)$ rounds with $O(tw_d)$ work.

PROOF. We can show using induction and Theorem 3.2 that graph G_j satisfies

$$(1 - \epsilon/\log \rho)^j G \preceq G_j \preceq (1 + \epsilon/\log \rho)^j G,$$

with probability $(1 - 1/n^2)^j$, and the expected number of edges in it is at most

$$O(st + m/2^j).$$

Since $j \leq \lceil \log \rho \rceil$, we get the desired spectral inequality. The parallel and distributed implementations are straightforward. The total work is dominated by the work performed in the first iteration, since the size of the graphs decrease geometrically. The same holds for the communication complexity. The claims on the parallel and distributed implementations then follow from Theorem 3.2. □

Notice that the algorithm can instead call HALFSPARSIFY-EXT and thus be implemented with $\lceil \log \rho \rceil$ passes over the input graph, in the distributed setting described in Section 3.2.

3.4. Sparsifying Bundle Decompositions

We now present a sparsification algorithm based on the observation that one can obtain tighter bounds on the effective resistance of the edges *contained* in the t-bundle that get progressively better as the size of the bundle increases. This is captured by the following lemma, which is a direct application of Lemma 2.4.

LEMMA 3.5. *Let $H = \sum_{j=1}^{t} H_j$ be a t-bundle decomposition of a graph G. Let the ith group of the bundle be the graphs*

$$H_{(i-1)k+1}, \ldots, H_{ik},$$

for $k = O(\log n)$. For each edge in the ith group we have $w_e R_e[G] \leq 1/i$.

In other words, for an edge e in group i, the product $w_e R_e[G]$ is upper bounded by $u_e = 1/i$. It is well understood that these upper bounds can be used to construct a $(1 + \epsilon)$-approximation of G with $O(T \log n/\epsilon^2)$ edges, where

$$T = \sum_e u_e = O(s \log n) \sum_{i=1}^{t/k} \frac{1}{i} = O(s \log n \log t).$$

Here s is an upper bound on the number of edges in spanner. This follows from a direct application of the "oversampling" lemma in Koutis et al. [2014], which can be parallelized and distributed as follows. Initially, each edge e of G is split into $(\lceil \epsilon^2/6 \log n \rceil)^{-1}$ copies of weight $w'_e = w_e \lceil \epsilon^2/6 \log n \rceil$. Then, each copy independently enters the sparsifier with probability u_e/T and weight $w'_e(u_e/T)^{-1}$. Multiple copies of an edge in the

sparsifier are then merged onto one edge by summing up their weights. This can be implemented in $O(1)$ parallel or distributed rounds by executing the splitting and merging implicitly. We thus arrive at the following theorem.

THEOREM 3.6. *There is an algorithm* SPARSIFYBUNDLE *where, on input, a t-bundle decomposition of a graph G on input G and ϵ outputs a graph $\tilde{G}$ that with probability $1 - 1/n^2$ satisfies the following:*

(*a*) $(1-\epsilon)G \preceq \tilde{G} \preceq (1+\epsilon)G$.
(*b*) *The expected number of edges in $\tilde{G}$ is at most $O(s \log^2 n \log t/\epsilon^2)$, where s is an upper bound on the size of the spanners in the decomposition.*

SPARSIFYBUNDLE *can be implemented in the CRCW PRAM to use work linear in the size of G and $O(1)$ time. It can also be implemented in the CONGEST model with linear communication complexity and $O(1)$ rounds.*

PROOF. Similar to the proof of Theorem 3.2. □

SPARSIFYBUNDLE can be used to further sparsify the output of SPARSIFY. In particular, when SPARSIFY is used with $\rho = O(n)$, we have $t = O(polylog(n))$. Using the spanner computation algorithm by Miller et al. [2015], we have $s = O(n \log\log n)$. So the output of SPARSIFYBUNDLE has $O^*(n \log^2 n/\epsilon^2)$[4] edges, which is the sparsest output known to be achievable with a combinatorial sparsification algorithm. Notice that SPARSIFY already expends the amount of work that SPARSIFYBUNDLE requires, so the extra sparsification step by SPARSIFYBUNDLE does not asymptotically increase the complexity.

4. IMPROVED PARALLEL SDD SOLVER

The Peng-Spielman parallel framework. Peng and Spielman [2014] gave the first solver for symmetric diagonally dominant (SDD) linear system that does near-linear work in polylogarithmic time. We shortly review the basic ideas behind their solver in order to highlight how our sparsification routine can be plugged into it, thus deriving work and time guarantees for a more efficient solver.

Let D be a diagonal matrix and A be the adjacency matrix of a graph with positive weights. The main idea in Peng and Spielman [2014] is a reduction of the input SDD linear system with matrix $M_1 = D - A$, to a linear system with matrix $\tilde{M}_1 = D - AD^{-1}A$ which is also shown to be SDD. Matrix $\tilde{M}_1$ is actually never formed explicitly because it can be too dense, as all vertices that are within a distance of 2 in graph A form now a clique in graph $AD^{-1}A$. The first step to remedying this problem is replacing $\tilde{M}_1$ with a $(1+\epsilon/2)$-approximation $\hat{M}_1$ that has $O(n + m \log n/\epsilon^2)$ edges and does not contain these cliques but replaces them with sparse graphs. As shown in Corollary 6.4 of Peng and Spielman [2014], this can be done in in $O(\log n)$ time and $O(n + m \log^2 n/\epsilon^2)$ work. The second step is further sparsifying $\tilde{M}_1$ down to $O(n \log^c n/\epsilon^2)$ non-zeros (for some fairly large constant c), using the parallelized Spielman-Teng sparsification algorithm. This step forms a matrix M_2 that is a $(1+\epsilon)$-approximation of $\tilde{M}_1$, and also an SDD matrix that is of the form $D' - A'$.

This construction is repeated recursively, producing an "approximate inverse chain" for M_1:

$$\{M_1, M_2, \ldots, M_d\}.$$

The depth d of the chain needs to be $O(\log \kappa)$ where κ is the condition number of M_1, that is, the ratio of its largest to its smallest non-zero eigenvalue. This is because, for

[4]We use $O^*()$ to hide $poly(\log\log n)$ factors.

$d = O(\kappa)$, the condition number of M_d is very close to 1, that is, M_d is essentially the identity matrix, and no further reductions are required. The $(1+\epsilon)$ approximations incurred by the construction of M_{i+1} from M_i compound in a multiplicative fashion. So, in order to keep the total approximation bounded, we need to pick, $\epsilon = \Theta(1/\log\kappa)$.

As shown in Theorem 4.5 of Peng and Spielman [2014], an approximate inverse chain can be used to produce an approximate solution for the system in $O(d\log n)$ depth and total work proportional to the total number of non-zero entries in the matrices that constitute the chain.

The improved solver. We now outline the construction of a parallel SDD solver that uses our improved parallel sparsification algorithm. We can think of all matrices in the approximate inverse chain as Laplacians, and we will refer to them as graphs. For simplicity, we will use $\tilde{O}$ to suppress polylogarithmic factors in n and κ. Also, we note that the spectral approximation bounds hold with high probability, and the claims on the number of edges of the sparsifiers hold in expectation.

Recall that in the construction of the approximate inverse chain, one has to set $\epsilon = \Theta(1/\log\kappa)$. Given that, observe also that the "threshold of applicability" of Theorem 3.4 is when the graph M_i has more than $\tilde{O}(n\log^2 n\log^2\kappa)$ edges, assuming that the sparsification factor ρ is of polylogarithmic size and using the parallel spanner computation by Miller et al. [2015]. Let us denote by m' this threshold. Whenever sparsification of $\tilde{M}_i$ is not possible, we simply let $M_{i+1} = \tilde{M}_i$, as implicitly done in Peng and Spielman [2014].

When constructing M_{i+1} from M_i, the number of edges increases by a factor of $O(\log n\log^2\kappa)$, in the first step that constructs $\tilde{M}_i$. In order to keep the total size of the inverse approximate chain and thus the work of the solver bounded, we only need to bring the graph back to its original size, if it exceeds m'. Besides its stronger guarantees, a relative advantage of our sparsification algorithm is that we can use it to sparsify the input graph by any factor ρ rather than aim for a very sparse graph as Peng and Spielman propose. So, using Theorem 3.4, the graph can be sparsified down to $O(m'+m)$ edges by setting $\rho = O(\log n\log^2\kappa)$. The total work is $\tilde{O}((m'+m)\log^3 n\log^4\kappa)$, dominated by finding an $\tilde{O}(\log^2 n\log^2\kappa)$-bundle in a graph of size $O(m\log n\log^2 k)$. Hence the total size of the approximate inverse chain is $\tilde{O}((m'+m)\log\kappa)$, and the total work required for its construction is $\tilde{O}((m'+m)\log^3 n\log^5\kappa)$.

We can improve the dependence on m by constructing the chain not for the input matrix M but for a 2-approximation M' of it, which has $\tilde{O}(n\log^2 n + m/\log^3 n\log^4\kappa)$ edges. This can be constructed by invoking Theorem 3.4, with $\epsilon = 1/2$ and $\rho = O(\log^3 n\log^4\kappa)$. The total work for this step is $\tilde{O}(m\log^2 n)$. It is well understood that this approximate chain for M' can be used as a preconditioner for M (in the same way its own chain would be used) incurring only a constant factor in the work and time guarantees.

Combining the above with Theorem 4.5 of Peng and Spielman [2014], we get the following theorem.

THEOREM 4.1. *On input of a linear system $Mx = b$, where M is an SDD matrix of dimension n with m non-zeros, a vector x' that satisfies $||b - M^{+}x||_M < \epsilon$ can be constructed with probability at least $1/2$ in polylogarithmic time and $\tilde{O}(m\log^2 n + m'\log^3 n\log^5\kappa)$ work.*

5. CONSEQUENCES IN DISTRIBUTED ALGORITHMS

Given any problem on a graph with m edges, a sequential algorithm can be easily turned into a distributed algorithm in the CONGEST model [Peleg 2000]. The network topology is collected in one node of the graph in $O(m)$ rounds of communication; the

node uses the sequential algorithm to solve the problem and then broadcasts it back to the network in another $O(m)$ rounds [Pandurangan and Khan 2010].

The first CONGEST algorithm requiring $o(m)$ rounds of communications for approximately solving the well-known sparsest cut problem and a number of other related cut problems was given in Sarma et al. [2013]. The algorithm works in $O((n + 1/\phi)\log^2 n)$ rounds, where ϕ is the conductance of the graph, that is, the value of its sparsest cut. The algorithm computes a cut with a value that approximates the sparsest cut within a factor of $O(\phi^{-1/2}\log^c n)$, where c is some constant.

Spectral sparsifiers preserve not only the quadratic form of the Laplacian but also several combinatorial properties of the graph. In particular, a spectral sparsifier is also a cut sparsifier, that is, a graph preserving within a factor of $1 + \epsilon$ the value of *every* cut $(S, V - S)$ of the graph, defined as the total weight of the edges with one endpoint in S and one in $V - S$. Thus, to approximately solve the sparsest cut problem, one can first sparsify the graph and then solve it on the sparsifier.

As observed by the authors of Pandurangan and Khan [2010], after the conference version of the present work appeared [Koutis 2014], the above approach can also be applied for the sparsest cut problem in the CONGEST model. Thus a by-product of the distributed sparsification algorithm is an algorithm that runs in $O(n\log^6 n/\epsilon^2)$ rounds and approximates the sparsest cut with a factor of $O((1 + \epsilon)\sqrt{\log n})$ using the state-of-the-art sequential algorithm for the sparsest cut [Arora et al. 2009] or within $(1 + \epsilon)$ if no restriction is imposed on the work performed at the nodes. This clearly improves not only the approximation quality but also the number of rounds for graphs with conductance smaller than $1/n\log^4 n$.

6. CONCLUDING REMARKS AND OPEN PROBLEMS

Remark 1. Multigrid algorithms provably do linear work in logarithmic time for certain very special classes of SDD systems that arise from the discretization of partial differential equations [Bramble 1993]. The algebra underlying multigrid differs considerably from that used by Peng and Spielman; in contrast with their algorithm, the spectral approximation does not accumulate multiplicatively in the multigrid "chain." This imposes a much less demanding constraint for the approximation quality between two subsequent levels, which can be constant, rather than $O(1/\log\kappa)$. Much of the efficiency of these specialized multigrid algorithms stems from this fact. It remain open whether something similar is possible for general SDD matrices. In particular, it is still open whether there is an $O(n)$-work $O(\log n)$ time algorithm for regular weighted two-dimensional grids that are "affinity" graphs of images. Experimental evidence seems to suggest that the possibility cannot be dismissed [Krishnan et al. 2013].

Remark 2. While a significant improvement over the solver was presented in Peng and Spielman [2014], the total work of the parallel solver remains high for sparse graphs, in terms of the logarithmic factors. We conjecture that further improvements are possible and will probably have to use a different algebraic framework (see Remark 3). Within the Peng and Spielman framework, it seems plausible that improvements can come from replacing the t-bundle by a sparser object; this presents us an interesting problem.

Remark 3. The complexity of solving SDD linear systems in the CONGEST model is an interesting and mostly unexplored problem. The simplest upper bound comes from using a common iterative method such as Conjugate Gradients iteration [Axelsson 1994]. Standard arguments can show that it requires $O(\min(1/\phi(\log(1/\epsilon), n))$ to converge to an ϵ-approximate solution in the sense used in Theorem 4.1. Designing an algorithm running in $o(n)$ rounds is an interesting problem. The algorithm in Peng and Spielman [2014] offers a possibility; the main technical difficulty lies in the

fact that the algorithm establishes new edges. While using these edges in the parallel model is simple, it probably requires sophisticated routing in the CONGEST model, assuming that the structure of this augmented graph allows for a routing with relatively low congestion. There is a lower bound of $O(\sqrt{n}+D)$ for the sparsest cut problem [Pandurangan and Khan 2010], where D is the diameter of the graph. Although the details of the reduction have to be worked out, it seems plausible that this implies the same lower bound for solving SDD linear systems as well.

ACKNOWLEDGMENTS

We thank Gopal Pandurangan for bringing interesting applications of distributed spectral sparsification to our attention.

REFERENCES

Sanjeev Arora, Satish Rao, and Umesh Vazirani. 2009. Expander flows, geometric embeddings and graph partitioning. *J. ACM* 56, 2, Article 5 (April 2009), 37 pages. DOI:http://dx.doi.org/10.1145/1502793.1502794

Owe Axelsson. 1994. *Iterative Solution Methods*. Cambridge University Press, New York, NY.

Surender Baswana and Sandeep Sen. 2007. A simple and linear time randomized algorithm for computing sparse spanners in weighted graphs. *Random Struct. Algorith.* 30, 4 (2007), 532–563.

Joshua D. Batson, Daniel A. Spielman, and Nikhil Srivastava. 2009. Twice-Ramanujan sparsifiers. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing*. 255–262.

Joshua D. Batson, Daniel A. Spielman, Nikhil Srivastava, and Shang-Hua Teng. 2013. Spectral sparsification of graphs: Theory and algorithms. *Commun. ACM* 56, 8 (2013), 87–94.

Erik G. Boman and Bruce Hendrickson. 2003. Support theory for preconditioning. *SIAM J. Matrix Anal. Appl.* 25, 3 (2003), 694–717.

James H. Bramble. 1993. *Multigrid Methods*. Chapman and Hall.

Peter G. Doyle and J. Laurie Snell. 2000. Random Walks and Electric Networks. (2000).

N. Harvey. 2012. Matrix Concentration. http://www.cs.rpi.edu/~drinep/RandNLA/slides/Harvey_RandNLA@FOCS_2012.pdf. (2012).

Michael Kapralov and Rina Panigrahy. 2012. Spectral sparsification via random spanners. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference (ITCS'12)*. ACM, New York, NY 393–398. DOI:http://dx.doi.org/10.1145/2090236.2090267

Jonathan A. Kelner and Alex Levin. 2011. Spectral sparsification in the semi-streaming setting. In *Proceeding of the 28th International Symposium on Theoretical Aspects of Computer Science, STACS*. 440–451.

Jonathan A. Kelner, Lorenzo Orecchia, Aaron Sidford, and Zeyuan Allen Zhu. 2013. A simple, combinatorial algorithm for solving SDD systems in nearly-linear time. *CoRR* abs/1301.6628 (2013).

Alexandra Kolla, Yury Makarychev, Amin Saberi, and Shang-Hua Teng. 2010. Subgraph sparsification and nearly optimal ultrasparsifiers. In *Proceedings of the 42nd ACM Symposium on Theory of Computing, (STOC)*. 57–66.

Ioannis Koutis. 2014. Simple parallel and distributed algorithms for spectral graph sparsification. In *SPAA*, Guy E. Blelloch and Peter Sanders (Eds.). ACM, New York, NY, 61–66.

Ioannis Koutis, Alex Levin, and Richard Peng. 2012a. Faster spectral sparsification and numerical algorithms for SDD matrices. *CoRR* abs/1209.5821 (2012).

Ioannis Koutis, Alex Levin, and Richard Peng. 2012b. Improved spectral sparsification and numerical algorithms for SDD matrices. In *Proceedings of the 29th International Symposium on Theoretical Aspects of Computer Science, STACS*. 266–277.

Ioannis Koutis and Gary Miller. 2009. The Combinatorial Multigrid Solver. (March 2009). Conference Talk.

Ioannis Koutis, Gary L. Miller, and Richard Peng. 2011. A nearly $m \log n$ solver for SDD linear systems. In *FOCS'11: Proceedings of the 52nd Annual IEEE Symposium on Foundations of Computer Science*. IEEE Computer Society.

Ioannis Koutis, Gary L. Miller, and Richard Peng. 2012. A fast solver for a class of linear systems. *Commun. ACM* 55, 10 (Oct. 2012), 99–107. DOI:http://dx.doi.org/10.1145/2347736.2347759

Ioannis Koutis, Gary L. Miller, and Richard Peng. 2014. Approaching optimality for solving SDD linear systems. *SIAM J. Comput.* 43, 1 (2014), 337–354. DOI:http://dx.doi.org/10.1137/110845914

Dilip Krishnan, Raanan Fattal, and Richard Szeliski. 2013. Efficient preconditioning of Laplacian matrices for computer graphics. *ACM Trans. Graph.* 32, 4 (2013), 142.

Vipin Kumar. 2002. *Introduction to Parallel Computing* (2nd ed.). Addison-Wesley Longman, Boston, MA.

Oren E. Livne and Achi Brandt. 2012. Lean algebraic multigrid (LAMG): Fast graph laplacian linear solver. *SIAM J. Sci. Comput.* 34, 4 (2012).

Gary L. Miller, Richard Peng, Adrian Vladu, and Shen Chen Xu. 2015. Improved parallel algorithms for spanners and hopsets. In *Proceedings of the 27th ACM on Symposium on Parallelism in Algorithms and Architectures (SPAA'15)*. ACM, New York, NY, 192–201. DOI: http://dx.doi.org/10.1145/2755573.2755574

Lorenzo Orecchia and Nisheeth K. Vishnoi. 2011. Towards an SDP-based approach to spectral methods: A nearly-linear-time algorithm for graph partitioning and decomposition. In *SODA*, Dana Randall (Ed.). SIAM, 532–545.

Gopal Pandurangan and Maleq Khan. 2010. Algorithms and theory of computation handbook. Chapman & Hall/CRC, Chapter Theory of Communication Networks, 27–27. http://dl.acm.org/citation.cfm?id=1882723.1882750

David Peleg. 2000. *Distributed Computing: A Locality-Sensitive Approach*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA.

Richard Peng. 2013. *Algorithm Design Using Spectral Graph Theory*. Ph.D. Dissertation. Carnegie Mellon University.

Richard Peng and Daniel A. Spielman. 2014. An efficient parallel solver for SDD linear systems. In *Proceedings of the 46th Annual ACM Symposium on Theory of Computing (STOC'14)*. ACM, New York, NY, 333–342. DOI: http://dx.doi.org/10.1145/2591796.2591832

Atish Das Sarma, Anisur Rahaman Molla, and Gopal Pandurangan. 2013. Distributed computation of sparse cuts. *CoRR* abs/1310.5407 (2013).

Daniel A. Spielman and Nikhil Srivastava. 2008. Graph sparsification by effective resistances. In *Proceedings of the 40th Annual ACM Symposium on Theory of Computing (STOC)*. 563–568.

Daniel A. Spielman and Shang-Hua Teng. 2004. Nearly-linear time algorithms for graph partitioning, graph sparsification, and solving linear systems. In *Proceedings of the 36th Annual ACM Symposium on Theory of Computing (STOC)*. 81–90.

Shang-Hua Teng. 2010. The Laplacian paradigm: Emerging algorithms for massive graphs. In *Proceedings of the 7th Annual Conference on Theory and Applications of Models of Computation (TAMC'10)*. Springer-Verlag, Berlin, Heidelberg, 2–14. DOI: http://dx.doi.org/10.1007/978-3-642-13562-0_2

Mikkel Thorup and Uri Zwick. 2005. Approximate distance oracles. *J. ACM* 52, 1 (2005), 1–24.

Joel A. Tropp. 2012. User-friendly tail bounds for sums of random matrices. *Found. Comput. Math.* 12, 4 (2012), 389–434.

Received August 2014; revised December 2015; accepted May 2016